Solution Architecture

Smart Lender | Team Lead: Venkat Rajarapu | Contributor: Kollipara TejaHansika

High-Level Architecture

User Browser --> HTML Frontend (Jinja2 templates) --> Flask Backend (app.py) --> Trained ML Model (model.pkl) --> Prediction Result --> back to User

Entity Relationship Overview

- Applicant (ApplicantID, Income, Education, EmploymentStatus, CreditHistory)
- LoanApplication (LoanID, ApplicantID [FK], LoanAmount, LoanTerm, PropertyArea)
- LoanDecision (LoanID [FK], Status, PredictionConfidence)

Component Breakdown

Layer	Component	Responsibility
Presentation	index.html / result.html	Collect input, display result
Application	Flask app.py	Route handling, request validation, calling the model
Model	model.pkl	Trained classifier (best of DT / RF / KNN / XGBoost)
Data (offline)	loan_data.csv	Source data for training, not used at runtime

Design Principles

- Keep the frontend lightweight — no heavy JS frameworks needed for the MVP.
- Load the model once at server startup, not per-request, for performance.
- Validate inputs on both client and server side.
- Keep preprocessing logic identical between training and inference to avoid skew.